

Submitting Your Anonymous Agent Report

*Instructor: Matthew Green**Applies to Assignments 1, 2, and 3*

This handout explains how to turn in the *agent report*. This is the short, anonymous write-up your coding agent produces about how well its teaching instructions worked. The tool, `tools/anon-report.py`, is one file of plain Python with no dependencies; you can run every step yourself, or let your agent do it (`AGENTS.md` contains these instructions as well.)

What it is, and why it works this way

The report is feedback on the *instructions* not on your performance. It tells us what the agent did versus what you did, where the “explain first” rule helped or got in the way, and what it would change. This file is not graded. We want it to be candid, so it’s anonymous — and rather than ask you to trust our promise on that, the system uses cryptography so that we can’t link a report to a student. If you’re interested, here’s what it offers:

- **Encryption.** Your report is encrypted on your own machine to the course public key. The drop box and the public bulletin board hold only ciphertext, and only the instructor’s offline key can decrypt it.
- **A ring signature.** Each report is signed with a *linkable ring signature* over the roster of registered student keys (your Part 0 key). It proves “an enrolled student wrote this” without saying which one, and it carries a tag that makes a second deposit from the same key visible as a duplicate, but without revealing the key. Tags differ per assignment, so your reports across the semester are not linkable to each other either.
- **Tor.** The deposit travels over Tor to an onion service, so the box never sees your IP address. The application keeps no per-request log and zeroes the stored file’s modification time; ordinary system metadata can still reflect when it arrived.

Later in the course you might implement this scheme yourself; it is the same math as the public-key block, used for a privacy-preserving and practical purpose.

What it doesn’t protect

Cryptography can offer only so much security. Filesystem or network records may reflect when you connect, and a distinctive writing style could narrow things down. We do not use that metadata to identify authors, but we will read the reports. The agent is instructed to write the report without names, JHED, email, file paths, verbatim code, or other identifying details, and to show it to you first. **Read the report before it is sent.** You’re the last check: you can edit it, although we’d prefer you not remove any interesting content.

Dates (Assignment 1)

Fri Sep 11	Part 0: register your public key (Gradescope “A1 Part 0”)
Mon Sep 14	Roster published: <code>anon-report/roster.json</code> appears in the course repository
by Sun Sep 20	Deposit your report
≈ Mon Sep 21	Tags snapshot published: <code>anon-report/tags-A1.json</code>
by Fri Sep 25	Attendance proof (Gradescope “A1 attendance”)

For Assignment 1 the channel is a **pilot**: the attendance proof is not required for review-lab sign-up. From Assignment 2 on, it will be.

Step by step

You can have your agent submit the report. Honestly, I would — the process is a bit annoying. If you choose to do it yourself, all of the instructions are below. Every command below is run from your assignment directory, on the machine that holds your Part 0 key (`~/.anon-report/key.json`).

1. **Get the roster.** On or about **Mon Sep 14** the course staff publish `anon-report/roster.json` in the course repository (<https://github.com/matthewdgreen/practicalcrypto2026>). It contains every registered key, the course encryption key, and the drop box’s onion address. Pull the repository (or download that one file) so your local copy is current. If the file isn’t there yet, the channel isn’t open.
2. **Before you deposit.** Have **Tor Browser** open and connected (free, <https://www.torproject.org>; the tool talks to it on port 9150 — a running **tor** daemon on 9050 also works).
3. **Read your report.** Have it as a text file, e.g. `agent-report.md`. Check it contains no name, JHED, email, paths, or code. This is the last moment it’s editable.
4. **Deposit it** — encrypts, signs, and uploads over Tor in one step:

```
python3 tools/anon-report.py deposit agent-report.md \
--roster anon-report/roster.json \
--assignment A1
```

Success looks like `ACCEPTED tag 3F9A...`. Note the tag: it’s the name of your entry on the public bulletin board. A receipt is saved under `~/.anon-report/receipts/`.

5. **If the upload fails** (typically “Tor not connected”), don’t re-run **deposit**. Fix Tor — on campus wifi, Tor Browser → Settings → Connection → Bridges → Snowflake — then re-send the saved receipt:

```
python3 tools/anon-report.py upload \
~/.anon-report/receipts/<newest file> \
--roster anon-report/roster.json
```

6. **One shot.** The drop box keeps the *first* deposit per key per assignment and refuses a second as a duplicate; there are no amendments. If staff publish a corrected roster, update it and deposit again: the server can replace only your same-tag entry from the superseded roster. Never use `--direct`: it skips Tor and shows the server your IP address.

7. **Optional: check your entry** once the board is published:

```
python3 tools/anon-report.py verify-bundle \  
anon-report/bulletin/A1/<your tag>.json \  
--roster anon-report/roster.json
```

8. **Attendance** (after `anon-report/tags-A1.json` appears). This proves you deposited a report without revealing which; your nonce is your JHED:

```
python3 tools/anon-report.py attend \  
--roster anon-report/roster.json --assignment A1 \  
--tags anon-report/tags-A1.json --nonce <your JHED>
```

Upload the resulting `attendance.json` to the Gradescope assignment “**A1 attendance**”, under your own name. If the tool says your tag isn’t in the list, your deposit didn’t land or the snapshot predates it — ask on Piazza rather than using `--force`.

9. **Keep things separate.** The report, the receipt, and `attendance.json` never go into your main assignment submission on Gradescope. Keep `~/anon-report/` for the rest of the semester.

Questions you might have

- *I didn’t use an agent.* Then there is no report to deposit for this assignment; nothing else to do.
- *I lost my key.* You can’t sign or prove attendance without it. Generate a new one with `keygen` and tell us on Piazza (privately); it goes on the next roster version. Back the key up this time.
- *Tor won’t connect on campus.* Use a bridge: Tor Browser → Settings → Connection → Bridges → Snowflake. Nothing else changes.
- *Can I see that my report arrived?* Yes — step 7. The board is public precisely so that everyone can check the box isn’t dropping deposits.
- *Can the instructor tell it’s mine?* Not from the system. See “What it doesn’t protect” for the honest residual: timing and writing style.
- *What does the server keep?* Ciphertext bundles, their ring signatures, and tags. No client IP addresses, plaintext, or per-request application log; ordinary system metadata may still reflect receipt timing.

Authorship and AI usage note (*in the same format this course asks of you*). This handout and the initial tool and server were written by **Matthew Green** with **Claude** (Anthropic’s Fable 5 model, via Claude Code). **OpenAI Codex** (the Daybreak Blue and GPT 5.6 Sol models) later performed a security review, hardening, adversarial and Tor end-to-end tests, and deployment verification. An early version used a wrong 2048-bit group prime; the current tool embeds the published RFC 3526 value and checks basic group invariants before use. Verify your constants.